

Session 05 - Long Short-Term Memory & GAN

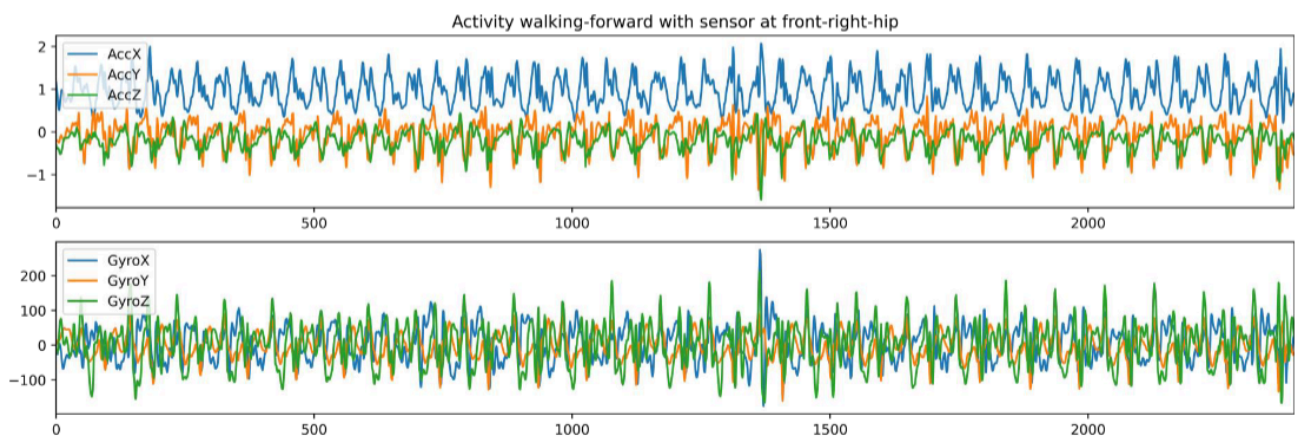
Recurrent Neural Network and Long Short-Term Memory

Model Layers

It's essential to **understand these two types basic neural network layers**:

- **Dense Layer**: Each input neuron is connected to every output neuron, making it suitable for general-purpose pattern recognition where input features are independent.
- **CNN Layer (Convolutional Neural Network Layer)**: CNNs are designed to build an understanding by analyzing neighboring inputs in an n-dimensional space. They are particularly effective for tasks involving spatial relationships, such as image processing.

Sequence Data

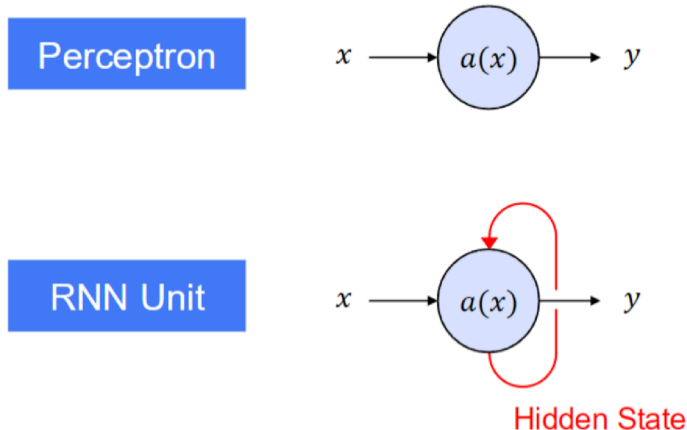


Graph showing a cyclical pattern indicative of walking motion.

Many real-world problems involve data where the order of elements matters. RNNs are specifically designed to handle such data, which includes:

- **Time Series Data**: Data points collected over time, where the sequence is crucial. Examples include sensor readings, stock prices, or physiological signals.
- **Natural Language**: Text and speech, where the meaning of words depends heavily on their context within a sentence or paragraph.

Recurrent Neural Network (RNN) Core Concept



Unlike feed-forward networks, RNNs have connections that loop back on themselves, allowing them to maintain an internal "memory" or **hidden state** that captures information from previous inputs in a sequence. This makes them suitable for sequential data where context from prior steps is essential.

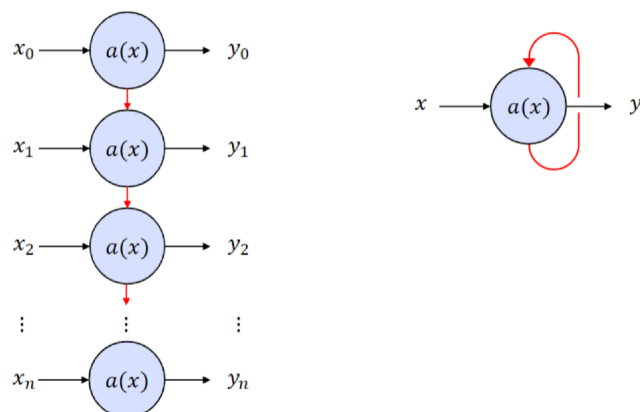
🔗 Difference Perceptron & RNN

The difference lies in this recurrent connection.

- A Perceptron takes an input x and produces an output y through an activation function $a(x)$.
- An RNN unit, however, also takes its previous hidden state (h_{t-1}) as an input, allowing information to persist across time steps.

Unrolled Recurrent Neural Network

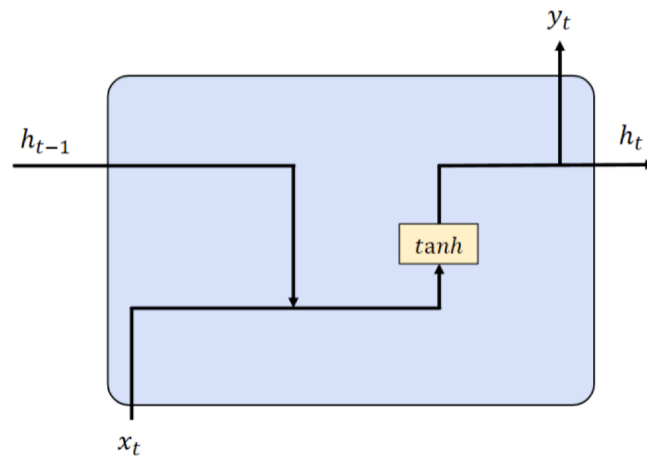
For an RNN to process a sequence, it can be "unrolled" over time. This shows a series of identical RNN units, where the output of one unit at time step $t-1$ becomes an input to the next unit at time step t .



For a sequence of inputs

$x_0, x_1, x_2, \dots, x_n$, the RNN produces corresponding outputs $y_0, y_1, y_2, \dots, y_n$, with each $a(x)$ unit passing information forward in time.

A typical RNN cell



At time step t the cell takes two inputs: the current input x_t and the hidden state from the previous time step h_{t-1} . It then computes a new hidden state h_t and an output y_t . The $\tanh$ activation function is often used within the RNN cell to introduce non-linearity.

Limitations of Simple RNNs:

While RNNs are good at processing sequential data, they suffer from a significant drawback: they struggle with

- **long-term dependencies.** As information propagates through many time steps, the influence of earlier inputs diminishes rapidly due to the
- **vanishing gradient problem.** This means that while each state receives "recent" information, information from much earlier in the sequence tends to get lost, making it difficult for the network to learn relationships that span long time intervals.

RNN in TensorFlow

A common way to implement RNNs is using frameworks like TensorFlow. The provided code snippet demonstrates a simple RNN model with two `SimpleRNN` layers:

```
# Define the input layer with shape (WINDOW_LENGTH, 3)
# This means a sequence of WINDOW_LENGTH time steps, each with 3 features
inputs = Input(shape=(WINDOW_LENGTH, 3), name="Input")

# Add the first SimpleRNN layer with 32 units
# 'return_sequences=True' ensures the entire sequence is passed to the next RNN layer
x = SimpleRNN(32, name="RNN_1", return_sequences=True)(inputs)

# Add a second SimpleRNN layer with 32 units
# This one returns only the final output in the sequence
x = SimpleRNN(32, name="RNN_2")(x)

# Add the output Dense layer with softmax activation
# The number of output neurons equals the number of classes
prediction = Dense(len(classes), activation="softmax", name="Output")(x)

# Define the full model using the functional API
model_functional = tf.keras.Model(inputs=inputs, outputs=prediction)

# Print the model architecture summary
model_functional.summary()
```

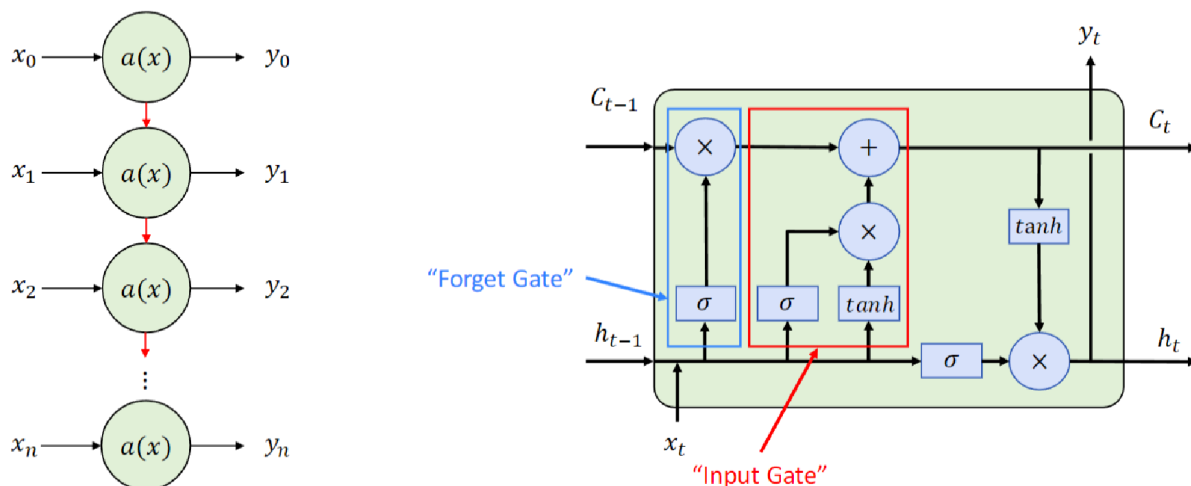
Long Short-Term Memory (LSTM) Networks

To address the limitations of simple RNNs, **Long Short-Term Memory (LSTM)** can help.

LSTMs are a special kind of RNN capable of learning long-term dependencies. They were proposed by Sepp Hochreiter and Jürgen Schmidhuber in 1997.

LSTM Architecture

LSTMs overcome the **vanishing gradient problem** by using a more complex internal structure called a "cell state" and several "gates" that control the flow of information. The cell state acts as a "highway" for information to flow unchanged through the entire sequence.



The main components of an LSTM cell are:

- **Forget Gate:** Decides what information to discard from the cell state. It looks at the previous hidden state (h_{t-1}) and the current input (x_t) and outputs a number between 0 and 1 for each number in the cell state C_{t-1} . A 1 means "keep this," while a 0 means "forget this".
- **Input Gate:** Decides what new information to store in the cell state. It consists of two parts: a sigmoid layer that decides which values to update, and a $\tanh$ layer that creates a vector of new candidate values (C_t) that could be added to the state.
- **Output Gate:** Decides what to output from the current cell state. It uses a sigmoid layer to decide which parts of the cell state
- C_t to output, and then puts the cell state through $\tanh$ (to push the values between -1 and 1) and multiplies it by the output of the sigmoid gate.

LSTM in TensorFlow

The provided code shows an LSTM model with two LSTM layers:

```
# Define the input layer with shape (WINDOW_LENGTH, 3)
# This represents a sequence of WINDOW_LENGTH time steps with 3 features per step
inputs = Input(shape=(WINDOW_LENGTH, 3), name="Input")

# Add the first LSTM layer with 32 units
# return_sequences=True means the entire output sequence is passed to the next LSTM
x = LSTM(32, name="LSTM_1", return_sequences=True)(inputs)

# Add a second LSTM layer with 32 units
```

```
# This one only returns the final hidden state (not the full sequence)
x = LSTM(32, name="LSTM_2")(x)

# Add the output Dense layer with softmax activation
# The number of neurons equals the number of classes to predict
prediction = Dense(len(classes), activation="softmax", name="Output")(x)

# Define the model using the functional API
model_functional = tf.keras.Model(inputs=inputs, outputs=prediction)

# Print a summary of the model structure
model_functional.summary()
```

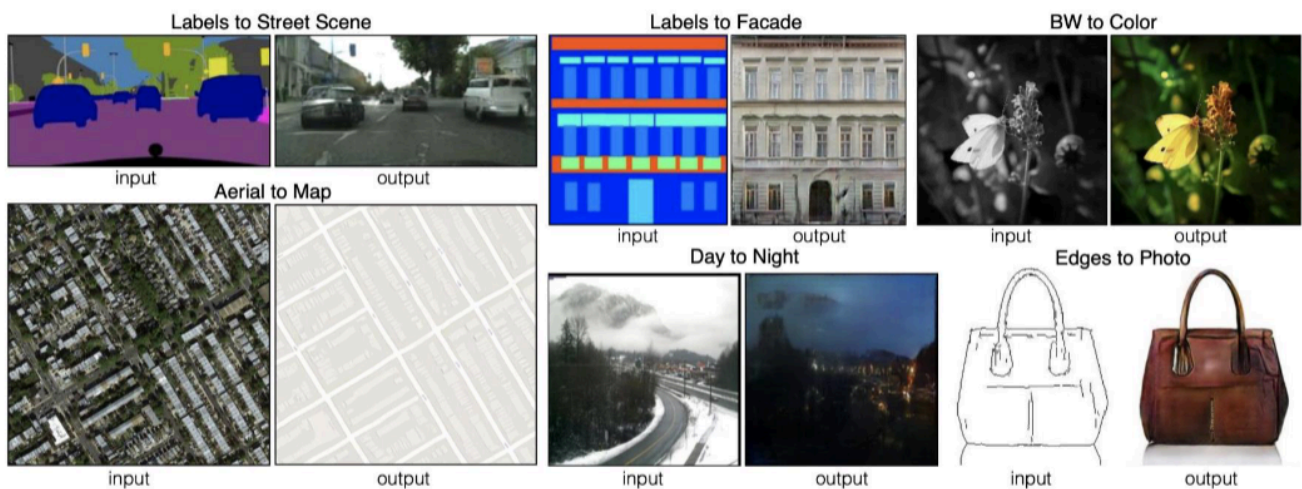
📌 How to keep track of complex patterns!

LSTMs (Long Short-Term Memory networks) include additional gates (**input, forget, and output gates**) along with a **cell state** that enables better control over what information to keep or discard. Each gate has its own set of weights, which significantly increases the total number of trainable parameters (**13,324 trainable parameters**), compared to SimpleRNNs (**3,628**).

This richer architecture allows LSTMs to **retain relevant information over longer sequences**, making them more effective at learning **complex temporal patterns**, especially when dependencies span many time steps.

Generative Adversarial Networks (GANs)

GANs involve a unique training setup with two competing neural networks: a **Generator** and a **Discriminator**. This adversarial process allows GANs to produce highly realistic outputs, ranging from synthetic images to new styles.



GANs are effectively mapping pixels to pixels based on learned relationships

Applications of GANs

GANs have revolutionized various fields due to their ability to generate diverse and high-quality data. Key applications include:

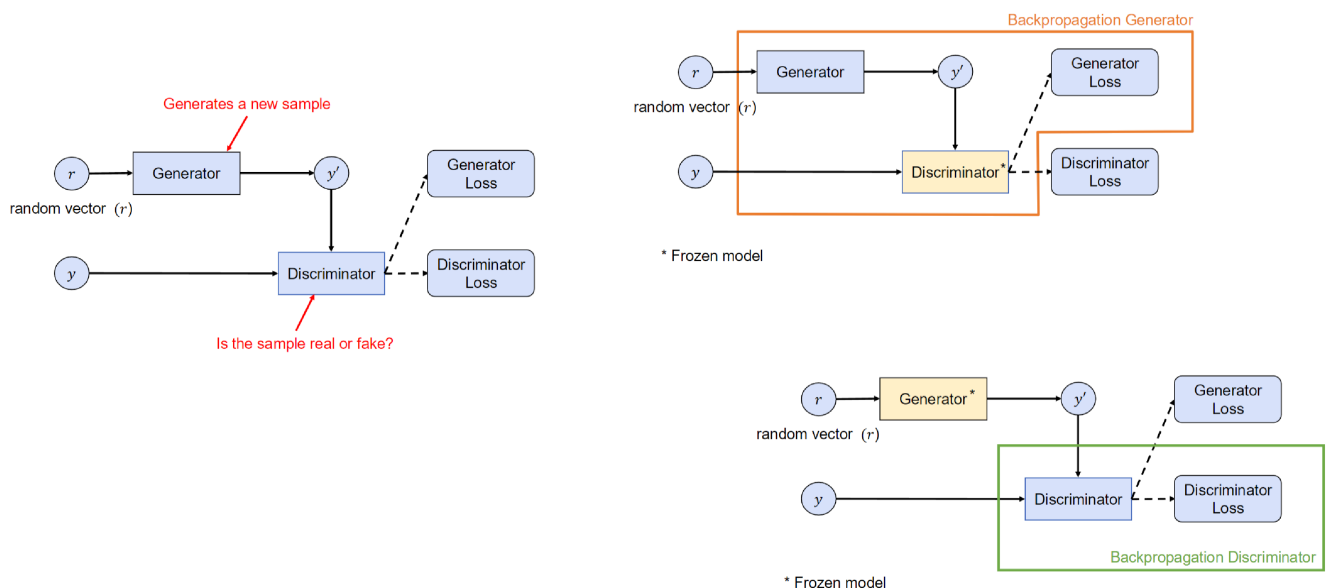
- **Create Data:** Generating new, realistic data points, such as images of human faces that do not exist.
- **Style Transfer:** Transforming the style of an image while preserving its content, or vice-versa.
- **Increase Image Resolution (Super-Resolution):** Enhancing low-resolution images to high-resolution ones, adding detail that was not present in the original.



Example of "Edges to Photo" is the edges2handbags tool, which allows users to sketch handbag outlines, and a GAN (specifically, a pix2pix model) then generates a photorealistic image of a handbag based on the sketch.

The Adversarial Process: Generator vs. Discriminator

The core idea behind GANs is a **minimax game** between two neural networks:



- **Generator (G):** This network takes a random noise vector (r) as input and generates a new sample (y') that attempts to mimic the real data distribution. Its goal is to produce samples so realistic that the Discriminator cannot distinguish them from real data.
- **Discriminator (D):** This network acts as a classifier, taking either a real sample (y) or a fake sample generated by the Generator (y') as input. Its goal is to accurately classify whether the input is "real" or "fake".

The Generator and Discriminator are trained in an adversarial manner, improving each other:

- **Discriminator Training:** During this phase, the Generator model is typically **frozen**. The Discriminator is trained to correctly identify real samples as "real" and fake samples (generated by the frozen Generator) as "fake." The Discriminator Loss measures how well it performs this classification.
- **Generator Training:** In this phase, the Discriminator model is typically **frozen**. The Generator is trained to produce samples that fool the Discriminator into classifying them as "real." The Generator Loss reflects how

successful it is at deceiving the Discriminator.

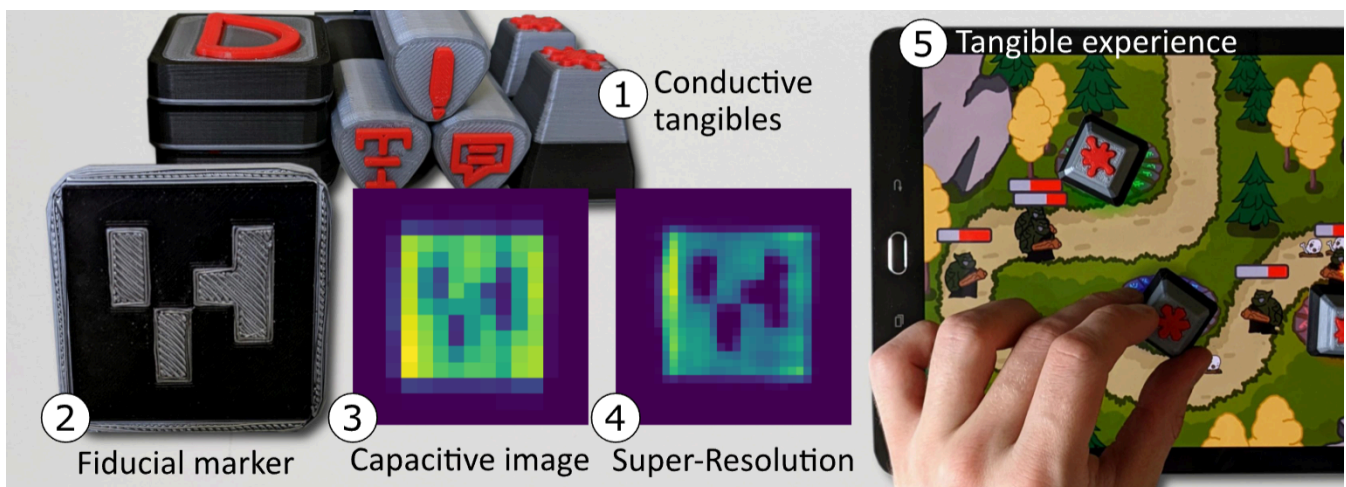
🔗 Training of GANs

"During the training process the generator and the discriminator improve each other by both getting better each trying to outperform the other model."

🔍 What makes a good Generator?

A "good" Generator is one that can produce samples that are indistinguishable from real data. Having **0.5 (or 50%)** would mean, that the Discriminator is essentially guessing, indicating that the Generator has become highly effective at producing convincing fake data.

Super Resolution with GANs



GANs are also highly effective for **Super Resolution**, a technique to enhance the resolution of images. This involves generating high-resolution (HR) images from low-resolution (LR) inputs, hallucinating details that were not present in the original.

This is explained quite well in detail in the section 1

